

总的来说, 层次化布局能够有效应对布局问题的复杂性, 同时保证布局约束行之有效, 并且有助于自动添加和合并保护环。由于层次化布局将电路分为多个模块, 这些模块可以独立设计并优化, 每一层只关注当前层的细节, 可以有效降低每次布局的复杂性。层次化布局能够在某一层次独立处理约束, 确保布局时约束实现的质量, 这对于减少失真和提高电路性能至关重要。

## 2.2 约束规则设计

按照网表规定, 布局的对象是实例 (Instance), 在本文的布局程序中, 将实例分为两种类型来考虑, 一种是底层器件, 另一种是非底层的模块。

布局器需要输入约束来指定受约束的模块和约束类型, 目前阶段本文布局程序的约束以文本形式输入, 考虑了 9 种约束类型, 设计了每种约束的描述形式, 具体规则如下:

(1) 共质心约束, 约束编号为 1, 描述格式为:

`groupName 1 deviceName1 deviceName2`

其中, `groupName` 代表设计者对约束组的命名, 1 表示共质心约束对应的编号, 这类共质心约束只可以作用于 2 个 MOSFET 器件, `deviceName` 表示器件名称, 需要注意器件的左右关系, 左侧的写在 `deviceName1` 的位置, 右侧的写在 `deviceName2` 的位置。

(2) 常规对称组约束, 约束编号为 3, 描述格式为:

`groupName 3 instanceName1 instanceName2 instanceName3 ...` (表示包含的器件)

`2 instanceName1 instanceName2` (表示两个 instance 对称)

`6 instanceName1 instanceName3...` (表示 instance 要求放在同一行, 可以无该约束)

...

对称组约束的约束对象是实例, 其中, 第一行指定约束组的名字、约束编号和所有约束组内包含的实例, 不限制约束组内出现实例的数量。对称组内有两种约束, 第二行的 2 表示对称关系对应的标识, 第三行的 6 表示对称组中放在同一行的约束对应的标识。由于对称组内必须有对称关系, 所以对称组中必须出现标识为 2 的约束描述, 但可以不出现标识为 6 的约束描述。组内不限制约束描述的数量, 可以存在多行 2 或者 6 的约束描述, 但要注意不能出现冲突, 例如一个器件 M0 只能和另一个器件 M1 对称, 不能再次出现和 M2 器件对称的情况。为了避免冲突, 本文的布局程序规定所有在约束组第一行出现的实例都必须存在至少一条约束描述, 并且若同一个实例出现在多个

约束描述中，不可以有两个标识为 2 的约束描述。对称约束需要注意左右关系，左侧的写在 instanceName1 的位置，右侧的写在 instanceName2 的位置。若 instanceName1 和 instanceName2 相同，布局器会将实例识别为自对称。

(3) 支持拆分器件的对称组约束，适用于电流镜，约束编号为 8，描述格式为：

groupName 8 deviceName1 deviceName2 deviceName3 ...

8 号约束组要求组内模块全部是 MOSFET 器件，deviceName1 对应的 MOSFET 放在中间，其他器件拆分放在两侧，除了中间的器件，其他器件数量 (m) 必须为偶数。具体的拆分模式会在第三章介绍。

(4) 宏模块邻近约束，约束编号为 4，不支持源漏共用，描述格式为：

groupName 4 deviceName1 deviceName2 ...

4 号约束组的组内成员都不是底层器件，表示将该组内模块单独提取出一个层次按照宏模块的方式进行布局，不限制模块的数量。

(5) 行对齐邻近约束，约束编号为 6，按行对齐摆放，支持源漏共用，描述格式为：

groupName 6 instanceName1 instanceName2 ...

6 号约束组内成员是实例，若是器件则支持源漏共用，不限制实例数量。

(6) 电容阵列约束，约束编号为 9，主要适用于电容阵列，目前支持的比例为  $1:2:4:\dots:2^k$ ，描述格式为：

groupName 9 instanceName1 instanceName2 ...

9 号约束组的组内成员可以是实例，其中的实例从左到右的比例必须符合从大到小的顺序，例如  $k=3$  时，描述为 C1<7:0> C2<3:0> C3<1:0> C4。

(7) 独立保护约束，约束编号为 7，表示某个器件需要被单独放置且包裹保护环，描述格式为：

groupName 7 deviceName

7 号约束组的成员一般来说是单一器件，也可以是实例，但在版图中一般不这样做。

(8) 旋转约束，约束编号为 11，用于指定某个模块特定的摆放方向，描述格式为：

groupName 11 instanceName rotateType

11 号约束组的组内成员是实例，一般是数字单元，其中，rotateType 为数字，指定摆放方向，符合表 2 中规则：

(9) BJT 阵列约束，约束编号为 5，主要适用于 BJT 阵列，描述格式为：

groupName 5 instanceName1 instanceName2

5 号约束组的组内成员可以是两个实例，要求比例为 1:k 其中 instanceName1 对应比例为 1 的实例，instanceName2 对应比例为 k 的实例，例如 k=8 时，两个实例依次为 Q1, Q0<7:0>。

表 2 模块方向类型表

| rotateType | 实际含义  |
|------------|-------|
| 0          | 不旋转   |
| 1          | R90   |
| 2          | R180  |
| 3          | R270  |
| 4          | MY    |
| 5          | MYR90 |
| 6          | MX    |
| 7          | MXR90 |

## 2.3 算法设计与实例

### 2.3.1 基于约束分层

与原理图对应，AMS 电路中有确切的层次结构，在真实层次的基础上，还可以引入虚拟层次来适应特定的电路需求，有助于缩小布局问题的规模。考虑 2.1 中介绍的 AMS 电路的特点，布局器可以根据约束类型将底层器件分为约束组，各自独立进行布局，目前考虑的约束主要有三种，分别是共质心约束、对称约束和邻近约束。

约束组可以包括多个嵌套层次，例如图 4 中展示了一个五管 OTA 结构，内部由三个约束组构成，其中差分负载是对称摆放的，差分对是共质心摆放的，电流镜按照电流镜约束摆放，这三个约束组还应按照同一个对称轴对称，所以他们还共同处于一个高层的对称约束组中。

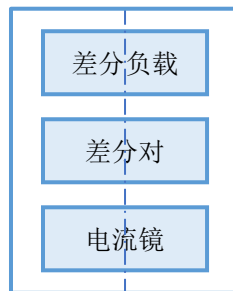


图 4 OTA 约束组示意图

在每个约束组内，调用相关约束适用的布局方法进行布局，在完成所有约束组内的

布局后,最上层的约束组被视作宏模块,对它们进行只优化线长和面积且考虑关键信号流的宏模块布局。考虑到这一点,我们将布局算法分为两个部分,一是底层针对约束的布局,二是顶层针对宏模块的布局。整个布局流程如图 5 所示,根据输入网表、约束文件获取布局层次,然后按层次提取信号流并布局,最后平滑层次,生成模块的最终位置坐标并写入到版图文件中。

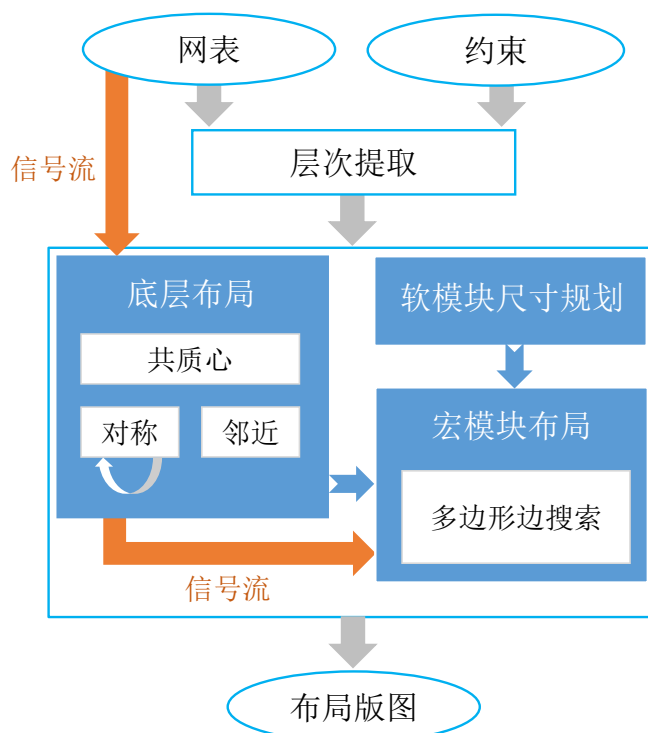


图 5 层次化布局流程图

### 2.3.2 构建层次树

在实际层次的基础上，我们插入虚拟层次，并将一个层次定义为一个组，每个电路层都由多个组组成，如果该层电路对应树结构中的一个节点，则其中每个组对应一个子节点。

按照这个规则，分析图 6 中的二级运放电路，可以生成图 7 中的层次树。其中 G0 对应电流镜结构，遵守对称约束；G1 对应差分对结构，遵守共质心约束；G2 对应差分负载结构，遵守对称约束；M6 是第二级源随器输入管，单独被一个保护环包围，因此独立构成一个组 G3。G0-G3 作为核心电路被合并为一个高层对称组 G4。其他的器件为控制电路的组成部分，每个器件可以作为独立的模块，也可以按照 MOSFET 的类型合并到一起，共享一个保护环，以节省面积。

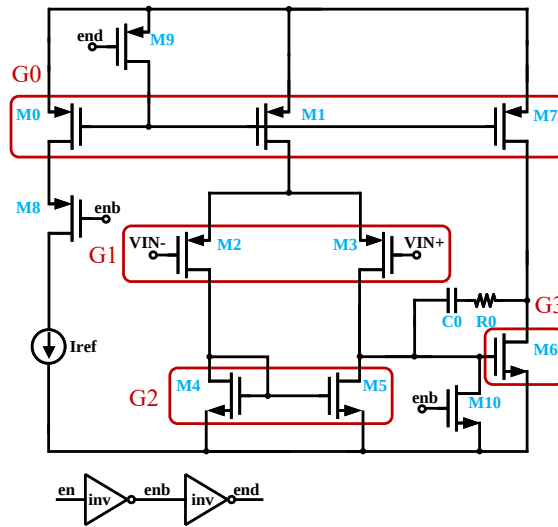


图 6 二级密勒运放电路图

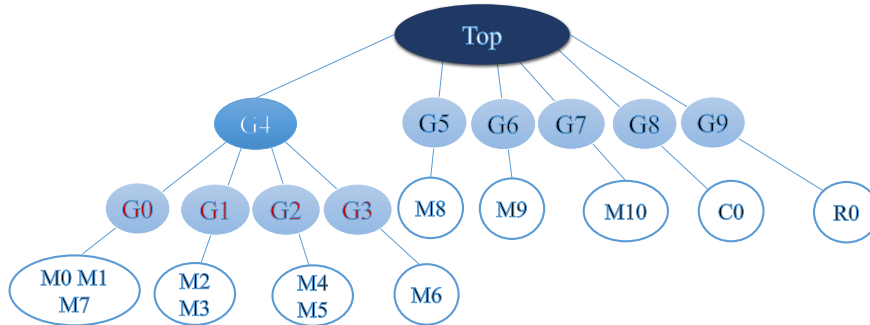


图 7 二级密勒运放层次树

## 2.4 基于引脚类型的信号流分析算法

注意到底层的器件本身具有的內部信号流关系和数字单元实例本身指定的引脚类型，在底层布局时，布局器可以通过引脚类型来自动分析器件（标准单元）之间的信号流关系。在完成底层信号流分析后，可以确定该层次与外界连接引脚的输入输出类型，从而达到自底向上传递信号流关系的目的。根据引脚类型，可以在每个层次构造出表示模块间信号流顺序的信号流关系图。需要注意的是模拟电路中存在反馈设计构成信号环路，这会导致信号流关系图中出现环路，算法需要自动检测环路的出现情况，并寻找合适的位置断开环路，以保证信号流顺序约束不出现冲突。

### 2.4.1 MOSFET 信号流

MOSFET 内部的信号流方向与电流方向一致，PMOS 电流从源极流向漏极，NMOS 电流从漏极流向源极，因此可以根据 MOSFET 的引脚类型分析纵向信号流流向。受原理图启发，将模块的引脚类型分为输入（Input）、输出（Output）和未知（InOut），整个电路的信号流关系是自底层传向上层的。